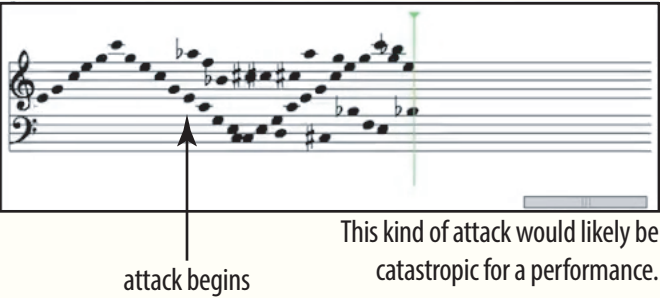


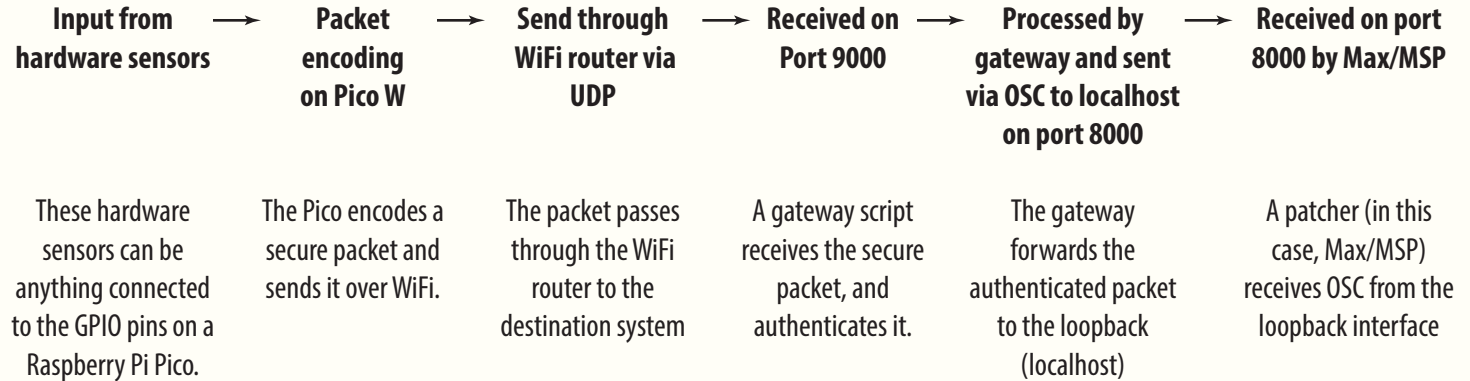
1.1 The Problem or Challenge

This project presents a lightweight authentication layer for transmitting OSC (Open Sound Control) packets over UDP, using HMAC and sequence-based replay protection. The approach is designed for resource-constrained embedded controllers that operate in live performance or real-time musical applications.

An attack on OSC: A victim transmitting OSC code that translates to a 3-octave C major arpeggio is attacked by a second device on the same network and port, transmitting random notes.



1.2 Secure System Overview



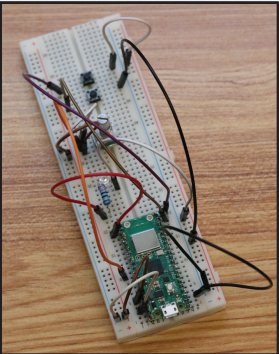
2. Secure Packet Design

On the secure system, the Raspberry Pi Pico W runs MicroPython code that reads values from a minimal musical control system consisting of a 10kΩ potentiometer and two buttons. It creates a 12-byte header, then appends the OSC string, and finally adds a 16-byte truncated HMAC-SHA256 authentication tag. It sends this packet through UDP over WiFi to the destination IP address at port 9000. The secure packet is constructed like this:



On the receiving end, a gateway receives the packet on port 9000, authenticates it, and then forwards it to the loopback (127.0.0.1) on port 8000. From here, Max/MSP (or, for that matter, anything else on the system) can get the authenticated OSC packet.

Secure system: Raspberry Pi Pico W with potentiometer, white status LED and two 6×6mm buttons



3. Attack Resilience

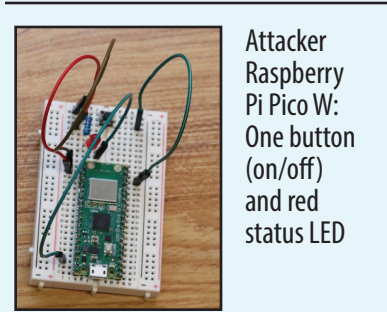
I subjected the secure system to three attacks from a separate Raspberry Pi Pico W, which was on the same WiFi network as the secure Pico. The first attack involved sending plain OSC packets with no header or HMAC tag. The second attack sent the right packet structure but with the wrong HMAC key. The third attack involved sending a captured packet that was out of order.

3.1 First Attack

The attacker sent regular OSC packets with no header or HMAC tag. The secure system rejected these and outputted the message DROP bad_length to stdio. Bad packets were not forwarded to the loopback.

3.2. Second Attack

The attacker sent a packet with the right structure (header, payload, HMAC tag) but the HMAC tag was generated with the wrong key. The secure system rejected the bad packets and responded with the output “DROP bad_hmac” to stdio. No bad packets were forwarded to the loopback.



Attacker Raspberry Pi Pico W: One button (on/off) and red status LED

3.3 Third Attack

The attacker somehow captured an authentic packet with the correct structure and properly generated HMAC tag. In this case, I simply replayed packets that the system captured with a diagnostic script I wrote, but a better hacker could potentially acquire packets by sniffing them. The secure system rejected the injected packet because its monotonic sequence number was out of order, printing the message “DROP replay” to stdio. Again, no bad packets were forwarded to the loopback.

4. Outcomes and Results

The system survived all three attacks. Provided integrity and authenticity but not confidentiality (no encryption). Latency was extremely low...